

East Alpha

11 July 2026 09:50

First off some commodities news that I might need to know, only basic, also including some quant shit like barrels of oil in the Strait and the price model etc.

Try to calculate price elasticity demand from 12 day war.
Do it with FactSet.

To-do:

Cover my CV

Stochastic modelling basics

Know about commodities, my quant project and the relation

A bit about East companies:

East Alpha: Quantitative commodity technology research company. Was the main brand.

East X: run the commodity strategies. Sit between pure systematic (pricing data models) and fundamental commodity players.

Multi-asset, multi-strategy across a diversified portfolio of contracts, targeting high long-term risk-adjusted returns, with low correlation to discretionary trading and other quant offerings.

Investment offerings/products:

East One; off the shelf curated total returns. Purely commodities, with the East X recommended portfolio mix.

East Values; runs same systematic strategy and quant framework as East One, with trading parameters reflecting the team values and views that include social and environmental issues.

East X Ventures — the deep-tech VC arm.

It sounds like East Alpha and East X are the same thing just do to with tax.

A big part about commodities is that it gives you exposure to inflation-sensitive instruments.

The commodities sector generally has a low correlation with other asset classes, offering diversification, although events such as the global financial crisis can overwhelm the supply/demand fundamentals that should dictate commodity prices. That trend has also been seen recently as trade tensions depress both stocks and commodities.

Should really lean into my current project, but look into some of the things I find with commodities, eg diversification/correlation and dispersion with the other asset classes, the index is diversified (check?).

Explaining my CV:

JP Morgan

JUST SAY THAT I HAVE TWO PROJECTS, ONE QUAL ONE QUANT.

Can also say that there was an opportunity to work on commodities quant strats so if this interview goes well I might take that offer (uncorr to CTA).

My quant project is:

In QS we have systematic signals, one of them is a CTA. Different strategies currently have a fixed vol budget as in x% of our risk budgeting goes to one strat, goes to another. We want to find out the following: Depending on the level of dispersion across asset classes but also within an asset class does that tell us anything about how we can

dynamically allocate to each strategy. For instance, I could smoothly lever up in high-dispersion regimes, and reduce risk when there is low dispersion and everything is moving in the same way which can go against us. Qual project: rolling up bellwether stocks. Later I might be looking to make new strategies, or realistically improve current ones.

ThemeBot, thematic NLP funds. Eg invest in the funds with highest scoring relations to themes like genetic therapies is a fund.

Notes:

EWMA is Exponentially Weighted Moving Average, a way to model and forecast vol.

EWMA assigns exponentially decreasing weights to older data, prioritizing the most recent price movements.

Today's variance:

$$\sigma_n^2 = \lambda \sigma_{n-1}^2 + (1 - \lambda) r_{n-1}^2$$

Today's variance = yesterday's estimated variance + the most recent squared daily return.

Lambda is the smoothing parameter or decay factor, 0 to 1 and how fast it forgets past vol. Lower lambda means past decays very fast, making the model more reactive. This way a model would react to new vol quickly, as opposed to equal weighted, but still quite smooth.

How the CTA works:

Loading closing prices and implied vols, derive log prices and log/arithmetic returns.

Builds a blended div_vol from EWMA realised vols and smoothed implieds, constructs four trend signals, regression trend t-stat, quantile (MA crossover), sign, volatility-breakout. Each averaged over multiple lookbacks.

The weights are 1/2 then else 1/6.

The main backtest loops over commodity, equity, bond, BCOM, vol-scales the target weights toward a target vol, day-by-day compounding AUUM, turnover, weekly rebal.

A rolling 10Y covariance matrix, ex-ante vol target, cap 7%.

The trend-following core of the CTA:

time-trend regression, rolling OLS of log price vs time, the slope of the t-stat (Newey-West) is the signal capped at ± 2 .

Sign/vol-scaled momentum where the sign of the cum return over the lookback, scaled by vol with a quadratic response.

Quantile/MA-crossover, short 10-day average vs the rolling median, above long, below short.

Volatility breakout is the price distance from iits moving average in standard deviations, breakout past 2 is full position.

The skewness is rolling return skew over 6/9/12 months, cross-sectionally ranked/z-scored within each asset class and sign-flipped. Go long the most negatively-skewed assets (those prone to sharp DDs) and short positive skew-ones, a contrarian bet that harvests the skew risk premium/mean reversion.

DD-recovery, DD_RC scores each asset by how far it sits from its rolling highs (DDs) and lows (recovery), vol-normalised, over 1-2 year windows, a recovery/mean-reversion tilt toward beaten-down names that are turning up.

Part of every strategy is inverse-vol position sizing to a target vol, asset-class neutralisation, weekly Wed rebal, turnover-based transactions costs and an ex-ante 7% vol cap using a rolling covariance matrix.

There are other strats written but not used.

Tanh-momentum, Holt-Winters double smoothing, statistically-meaningful trend trades when $R^2 > 0.65$ and $|t| > 2$.

Wellington Management:

Multi-asset team etc.

What I did in the oil short trade.

During the 12-day war, I labelled previous oil shock events, supply at risk x duration.

What I did:

Cover the other lines in the wellington part.

12-day war breakdown:

Initial strikes were on early Friday, so this is what I would've done on the Mon.

At the time we fundamentally had the idea that this wasn't going to last long.

Scenario analysis, assigning probabilities to different potential outcomes.

Global demand was 100mb/d, Iran exports 1.7mb/d, Hormuz 20mb/d

Pre-war June base was \$64, that Friday close was \$74, so \$9 (15%) risk premium already in the price.

Then in the case of the Strait closing

Calculated deficit of barrels:

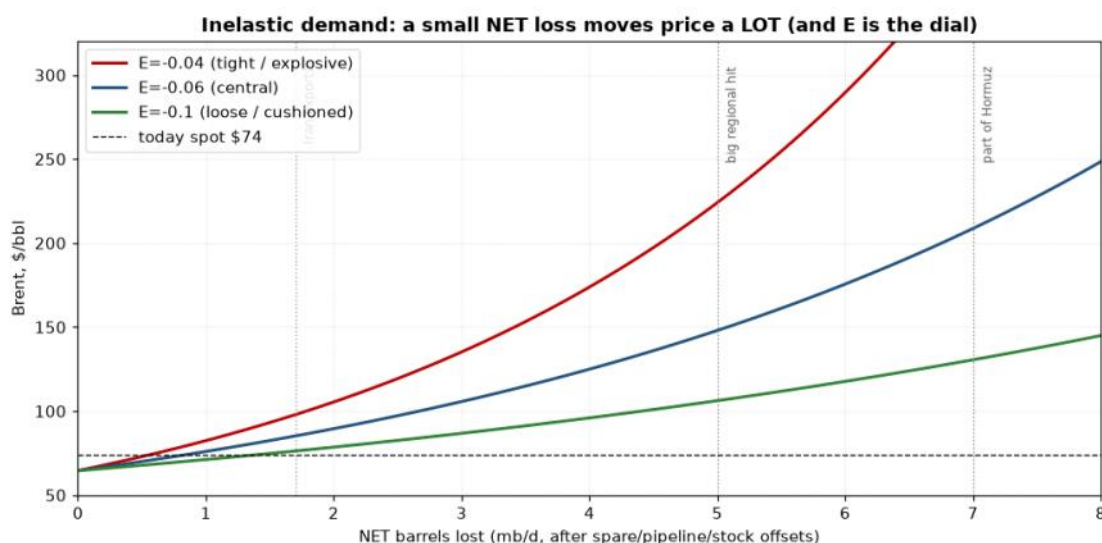
	item	mb_d
0	Global liquids demand	103.0
1	Iran crude output	3.3
2	Iran crude EXPORTS (mostly China)	1.7
3	Strait of Hormuz throughput	20.0
4	OPEC+ spare capacity	-4.5
5	Hormuz-bypass pipelines (usable)	-3.5
6	IEA/SPR sustained release	-2.5

	event	lost_mbd	world_supply	price_jump_pct	persistence	supply_lost_pct	apparent_E
0	Abqaiq drone strike, Sep-2019	5.7	100.0	14.6	temporary (restored ~2wk)	5.70	-0.390
1	Iraq invades Kuwait, Aug-1990	4.3	66.0	110.0	persistent (months)	6.52	-0.059
2	Libya civil war, 2011	1.6	88.0	15.0	persistent (months)	1.82	-0.121
3	Iran revolution, 1978-79	4.0	63.0	150.0	persistent + hoarding	6.35	-0.042

Short shocks had high implied PED.

A persistent shortage had low PED.

A 1% net supply loss (about 1mmb/d) at E=-0.06 (central case) lifts brent 18% (\$65 -> \$78).



PED of -0.04, -0.06, -0.1 from explosive to cushioned/resilient. These numbers came from previous oil supply-side shock events.

In the price engine what actually bites is $1/E$. At $E = -0.06$, $1/E \approx -16.7$, so:

$$P_1 = P_0 \left(1 - \frac{\text{net loss}}{\text{demand}}\right)^{1/E}$$

We were operating on the facts that the Strait had never been closed before, even during the Tanker war. Determined that during strikes oil would still flow, reduced, but at a risk premium.\W

	scenario	prob	gross	offset	net_lost	fear	model_px	brent	band_lo	band_hi
0	De-escalation / ceasefire	28%	0.0	0.0	0.0	-0.03	63.0	63.0	60	70
1	Contained conflict (base)	34%	0.0	0.0	0.0	0.14	74.0	74.0	70	82
2	Iran export / infra hit	20%	2.0	0.5	1.5	0.12	92.0	92.0	82	105
3	Regional spillover	11%	4.0	1.0	3.0	0.15	122.0	122.0	95	128
4	Partial Hormuz disruption	5%	13.0	6.0	7.0	0.20	157.0	157.0	115	165
5	Full sustained Hormuz close	2%	18.0	6.0	12.0	0.30	290.0	250.0	150	250

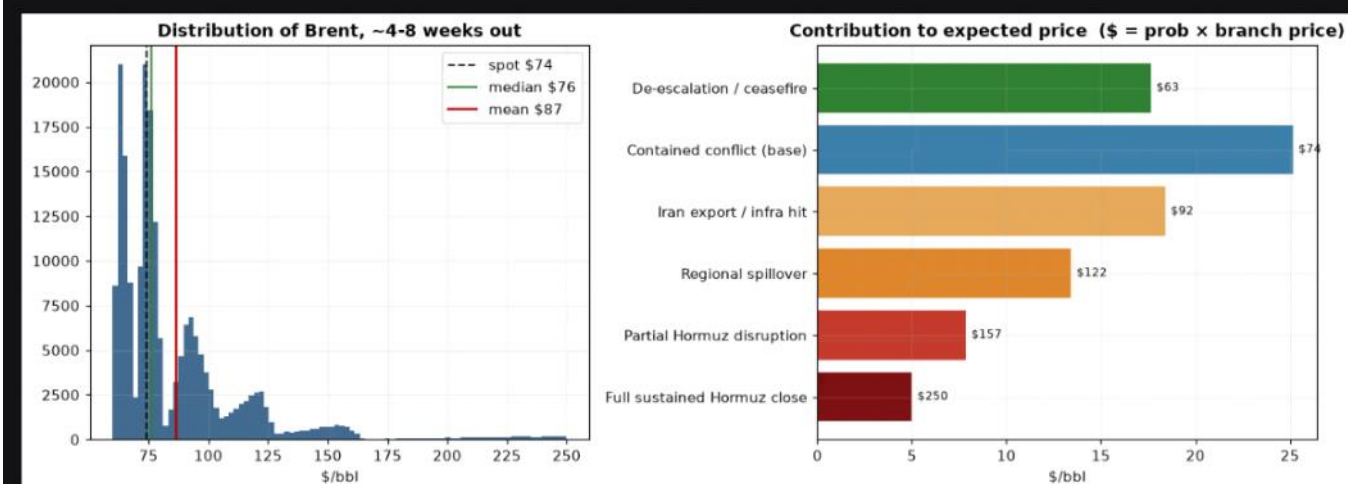
Probability-weighted EXPECTED Brent (~4-8 wks out): \$87.5 (spot \$74.2)

=> the tree implies \$+13.3 of average upside vs spot – driven by convexity, not the base case. Decompose it:

- 'no real disruption' sub-world (de-escalation + contained only) -> \$69 (< spot)
- the two Hormuz branches are just 7% of probability but add \$12.9 to the expected price (15% of it).
- spot \$74 sits BETWEEN the calm world (\$69) and the full-tree EV (\$87) -> on Day 2 the market was pricing a modest, non-trivial disruption odds.

Monte Carlo sims:

Expected (mean) Brent : \$ 86.5
 Median Brent : \$ 76.2 (vs spot \$74.2) <- most-likely path
 5 / 25 / 75 / 95 pctl : \$62 / \$67 / \$95 / \$144
 Tail probabilities:
 P(Brent > \$ 80) = 39.3%
 P(Brent > \$ 90) = 32.5%
 P(Brent > \$100) = 19.4%
 P(Brent > \$120) = 10.3%
 P(Brent > \$150) = 4.1%
 P(Brent < \$ 70) = 28.1% (premium unwinds / de-escalation)



The goal here is to turn the six discrete scenarios into a full probability distribution of Brent, so we can read off percentiles and tail probabilities instead of one point estimate, repeated 200k times.

Pick one of the six at random, weighted by probability assigned, over 200k draws you literally get the sampling scenarios tree. Draw a price within that scenario (since the price is a range between band_lo, brent (headline) and band_hi) from a triangular distribution, the three points. You have a bounded shape that peaks at the headline.

Lognormal risk-neutral $P(\text{Brent} > K)$ ($F=\$74$, $T\sim 6\text{wk}$), at two vol levels:

impl_vol	35%	50%
strike		
90	4.6	11.1
100	0.5	3.3
120	0.0	0.2
150	0.0	0.0

Our scenario tree says:

$P(> \$90) = 32.5\%$ (tree)
 $P(> \$100) = 19.4\%$ (tree)
 $P(> \$120) = 10.3\%$ (tree)
 $P(> \$150) = 4.1\%$ (tree)

Read: a plain lognormal UNDER-prices the far tail ($>\$120/\150) vs our tree, because it has no fat geopolitical JUMP. The real options market compensates with a steep upside call skew (25-delta risk-reversal bid for calls) – qualitatively the SAME fat right tail our tree produces. If live 25d-RR / vol were pulled (e.g. from FactSet or Bloomberg), you'd fit the jump size & probability to the quoted call skew and reconcile the two exactly. The takeaway is directional agreement: buy convexity, not flat length.

Overall thought that the Strait was not going to close, we were going to be resilient, and when the Brent price peaked we took an underweight/short.

Then just say I did a KNN strategy, using the most similar moves to the one that had just happened, to predict the next week and month. Week was a large price, month was high but less.

Do note though that in the current Iran situation:

Strait does 20mb/d, Saudi and UAE pipelines about 8mb/d. Reduced ships getting through getting about 5mb/d and China appearing to use reserves are not importing their 5mb/d. Hence there is only about 2mb/d missing as a result of the current situation.

For the spreads research:

CDX NA IG is an equally-weighted basket of 125 North American investment-grade CDS names, rolled every six months (March/September) into a new on-the-run series. Buying protection = short credit = you profit when spreads widen.

Note that there is a fixed 1% premium, and then an upfront cash payment is required on top to balance the trade. Eg if the market is riskier, the fair spread is above 100bps, so the buyer of protection pays the fixed 100bps plus an upfront fee to the seller.

I would do fundamental research on corporate fundamentals, eg earnings and revenues revisions and ratios. Default and downgrade probability. Aggregate net debt/EBITDA and EBITDA/interest expense across the IG universe. Interest coverage is the key one right now, because a lot of IG debt was issued at 2-3% and refinances at 5-6%. Track the maturity wall — how much IG paper rolls in the next 12–24 months, and at what spread pickup.

Decompose: spread $\approx$ expected loss + liquidity premium + risk premium.

Growth expectations, interest rate path, inflation. Financials willingness to lend.

Had models bang.

Breaking down CPI:

Eg in July 2025 CPI was quite high due to medical care index 0.7% over the month, dental 2.6% increase in July. The very next month they lowered down, mean-reverted, and can talk about how for the next print we incorporated those to lower them.

--

Fed signalling that I wrote:

Here I am actually just going to rip the Turnleaf Imperial strat/lecture and say I did it.

Three parts.

1 We had macro data, market data (eg pricing, implied vol), benchmark data, alt data like pollution time series as a proxy for industrials activity. The inflation model forecasts 1 to 12 months ahead, updated per country once a month shortly after that country's inflation release. Growth models target things like ISM; labour models target unemployment and non-farm payrolls. The 1-12 month forecast gives us a curve.

2 DM FX strategy

Three signals tested across a G10 cross set.

Signal 1 was the Reuters news sentiment where we aggregate country news into a sentiment index per currency. Take our news corpus, tag each item to a currency, only kept stories containing about a country. Ran a 1 hour return after the news. Scale by trailing volatility. Eg if we are in a generally volatile environment then a vol move might not be too irregular.

Three-class: top tertile = positive, bottom = negative, middle = neutral. Drops the ambiguous middle, which is mostly noise.

Baseline — TF-IDF. Build a vocabulary of words and bigrams across the corpus. Each headline becomes a sparse vector: one slot per vocabulary term, valued by term-frequency $\times$ inverse-document-frequency.

Logistic regression on the TF-IDF matrix. For each term you learn a weight w ; for a headline you compute $z = \sum w_i x_i + b$, then squash through a sigmoid to get a probability. Training adjusts the weights to minimise log loss, which punishes confident wrongness savagely.

Regularise hard. $C=0.1$ (strong L2) is not optional.

Then read the word rates and they should be things like hawkish, beat, or miss.

Split by time, never randomly. Train on 2015–2021, validate 2022, test 2023–24. Random k-fold leaks the future into the past: two wire updates about the same event, minutes apart, land on both sides of the split, and your model looks brilliant.

Or the keyword lexicon way:

Have a list of bullish terms, have a list of bearish terms, keep a list of negations, then depending for fixed income of fx it was either positive or negative.

For every term (unigram + bigram) appearing at least, say, 20 times in the corpus, compute the mean standardised forward return of the headlines containing. That way don't have to hand select every word. Then average the outcome of each headline which the word appeared to get a score. Then sum the scores to get the countries with the highest and lowest scores, then buy and short respectively.

Signal 2 within DM FX was to buy the currency with the higher front-end inflation forecast and sell the currency with the lower front-end inflation forecast. But not for long-term since inflation long-term can lose trust in that central bank's ability to keep financial stability.

Say that this was the Fed sentiment signalling part, reacting to the news headlines of the Fed, not the minutes that was already another model.

Long end rates had surged because of political uncertainty in the UK, quite high UKK inflation at the time 3.8%, QT added supply pressure, risk premium. Nov 26 budget was the event-driven risk premium, the budget landed well, rebuild fiscal headroom, yields fell.

Germany was the mirror image where bunds were being repriced cheaper on supply, debt-brake reform had unlocked large defence and infra programme, materially more Bund issuance into 2026, ECB done cutting but UK was pricing in a cut, so while UK was retiring the risk premium, Germany was building one. Were long gilts, short bunds, a spread tightener, which worked.

=====

What I did at Maven:

While I was at sixth form, B-S modelling prices, workshops, shadowing OMMs.

Worldquant:

Momentum strategies, using t-stats, correlations, longing the top companies of different metrics, eg

Longing companies where analysts were raising their earnings and revenues revisions.

Options implied call/put skews and vol spreads.

Datasets on social media sentiment and volume of posts about a company.

Simple price, volume strats

Trading on insider trades.

Neutralising across either market, industry, subindustry, or sector.

The options skew strats: If call IV is high relative to put IV, the market may be pricing more upside

demand/speculation. If put IV is high relative to call IV, it may reflect downside fear or hedging pressure.

Trade in the direction being priced in.

Would then rank them and long the stocks with high call-minus-put skew, short stocks with low or negative skew,

where puts are expensive vs puts. Basically betting that upside option demand contains useful positive info. We

would long the stocks with high upside pricing.

Did that across different expiries.

=====

RAIC research project:

RAIC, Responsible AI Consortium, is founded by a few partner universities, including Imperial College London, and companies file an AI problem that they need researchers to solve and students can be selected to take part.

For me, I was on a project with a biogas and environmental consulting company called Sureco who research but also implement biogas digesters across Indonesia.

Here they wanted a chatbot to automate helping farmers with their questions. First we made a decision tree chatbot where using Sureco's own troubleshooting, installation guide and biogas system user manuals and the farmer would select through decision trees what the issue is that they are having and how to fix it.

They wanted an actual chatbot like chatgpt but didn't want the costs of having a frontier lab model just answering farmer's questions, so we made an alternative chatbot.

RAG changes what the model knows.

Biogas plant specs, subsidy rules, local regulations, product docs — facts that change and need citing.

What was changing: the biogas equipment themselves, locations, CPOs (Construction Partner Organisations) and who to refer to were often changing, eg getting the most recent change of the CPO.

Fine-tuning was done to change output format, refuse any out of scope questions, was used to respond in different languages and dialects (refuse that I did not do that). We used an open-source Llama 4 (maverick) model.

Actual step-by-step:

Go to Hugging Face, where the weights are gated, request access and auth a token.

Start from the Instruct variant (just nudging style not teaching a conversation from scratch).

Size 8B for a single GPU.

Data: We gave it instruction-response pairs, rendered through the model's own chat template.

Say 500 examples created mostly through AI, using actual limited examples Sureco had and then the majority were new, so like bootstrapped in a way and we reviewed a lot of them.

Hold out a validation set.

Mask the prompt tokens so loss is computed only on the completion.

NF4 — a 4-bit datatype that's information-theoretically optimal for data that's normally distributed, which neural net weights approximately are. Not naive uniform int4.

Double quantisation — the quantisation constants themselves get quantised, saving ~0.4 bits/param.

Paged optimisers — NVIDIA unified memory pages optimiser state to CPU on memory spikes, so you don't OOM on a long sequence.

The base weights are frozen and quantised; they're dequantised to bf16 on the fly for each matmul. Nothing 4-bit is ever trained.

The maths: for a frozen weight W ($d \times k$), you learn $\Delta W = BA$ where B is $d \times r$ and A is $r \times k$, with $r \ll d$. The update is applied as $W + (\alpha/r) \cdot BA$. A is init'd Gaussian, B init'd to zero, so training starts as an exact no-op on the base model. You train only A and B — a few tens of millions of params instead of 8 billion.

r = capacity of the update. 8–16 for style/format tasks; 32–64 if you're pushing genuinely new domain behaviour. Higher r overfits faster on small data.

α = scaling. Convention is $\alpha \approx 2r$.

target_modules — the QLoRA paper's finding is that adapting all linear layers (attention and MLP) matters more than using a high rank on attention only. If you only did q/v , say so and say why (memory).

To train, use TRL's SFTTrainer, reasonable defaults.

1-3 epochs, more than that on a few thousand examples and you memorise.

Watch train vs validation loss. The classic QLoRA failure is train loss diving while val loss turns up after epoch 1 — stop there.

Merge or serve. Two options.

Serve: Keep the adapter separate (50-200MB) and load it on top of the base at inference.

Merge: (merge_and_unload) into a single set of weights for simpler deployment and zero added inference latency.

Note the subtlety: you merge into the dequantised fp16 base, not the 4-bit one — merging into the quantised weights degrades quality. Then re-quantise the merged model afterwards (GGUF for llama.cpp, or AWQ/GPTQ) for deployment.

For a "lightweight chatbot for an Indonesian non-profit" the natural end state is: merge → convert to GGUF → quantise to Q4_K_M → serve on llama.cpp/Ollama on cheap CPU hardware, with the RAG retriever in front. LoRA is quite resistant to catastrophic forgetting as you are freezing the base weights.

After all of that, funnily enough over a few hundred in a validation set the RAG outperformed and was generally easier to scale if new documents come in, if equipment changes, just add it to the vector database and you are good.

Why RAG over the QLoRA:

QLoRA can just hallucinate, interpolate over half facts, sound confident but wrong, quite tough to inject facts.

Say that there was RAG + Llama model where RAG was to get the facts and do the knowledge work and the fine-tune was there to keep it in boundaries, stay in the target languages, stay in format, stay grounded and refuse when retrieval came back empty.

One thing to be ready for: "how would you have known if the fine-tune wasn't adding anything?" The answer is the ablation — base + RAG vs tuned + RAG, on the same held-out set, scored on format adherence, language correctness, groundedness and refusal rate separately rather than one blended score.

Yield, ranking:

Specific methane yield = $\text{m}^3 \text{CH}_4$ per kg volatile solids (VS) fed. Per kg VS, not per kg wet manure — manure is mostly water and the water fraction varies wildly with how much the household dilutes the slurry.

Organic loading rate (OLR): $\text{kg VS} / \text{m}^3 \text{ digester} / \text{day}$ — are they overfeeding?

Hydraulic retention time (HRT) = $\text{digester volume} / \text{daily feed volume}$ — are they flushing the bugs out faster than they reproduce?

Volumetric gas production rate: $\text{m}^3 \text{ gas} / \text{m}^3 \text{ digester} / \text{day}$ — capacity utilisation.

Actual yield ÷ theoretical yield (BMP, or the Buswell equation from feedstock composition). This gives you a bounded 0–1 efficiency score, which is much nicer to rank on than an unbounded one.

So residualise first: regress yield on ambient temperature, feedstock type, digester size/design, age, altitude, season, herd size — then rank the residual. Now ranking build quality.

Rank within-unit too. Each digester's own history is the best baseline for itself. Cross-sectional rank tells you who's bad; within-unit z-score tells you who's getting worse, which is what you want for maintenance.

pH is a lagging indicator of digester souring. When feeding gets unbalanced, volatile fatty acids (VFAs) accumulate — but the bicarbonate alkalinity buffers them, so pH stays flat. By the time pH visibly drops, the buffer is exhausted and the digester is already dying; recovery takes weeks.

The leading indicator is the VFA:alkalinity ratio (FOS/TAC) — it rises while pH is still perfectly normal. Above roughly 0.4 you're in trouble; 0.3–0.4 is a watch zone. Even earlier: rising H_2 partial pressure, which signals that the syntrophic acetogens and methanogens have fallen out of balance.

You also have stove hours used, herd size of biofuel, weather data, days since install.

=====

OCR -> NLP:

A lot of these farmers have poor internet which is not super consistent, nor power. And so most of the work they do is handwriting. This tool was made only for those people.

Told the farmers must write in print handwriting or if they have something messily written to rewrite it.

We only made this because as Sureco scale across Indonesia, right now they only have a few hundred biogas digesters, but as they ramp up to thousands, and while they had us for the few months of the project, also got us to read text.

These were post-it notes that farmers would right on or a piece of paper or a technician comes around and these are local communities, and the owner of the biogas digester might not have been there so they write down briefly what the issue was, how they fixed it.

Also need blur detection.

Image preprocessing:

photo → find the page → dewarp → binarise → crop each field by coordinates → run Tesseract per field with the right config → validate

Install Tesseract.

Print four ArUco markers in the corners of the form. Detection is then near-perfect and gives you an exact perspective transform. The payoff is huge: every scan is now the same size with the same geometry, so field positions are fixed pixel coordinates. What you hand to Tesseract is always the same dimensions and centred on the text.

Then binarising, which is to turn the picture into just black and white, like literally either 0 or 255 and nothing in between. This strips out colour gradient when you take a picture which Tesseract needs, just black on white.

Then crop so that the box of text is always in the same coordinates, that way Tesseract doesn't have to do the finding.

Run Tesseract. It also has an Indonesian dictionary.

Anything that comes back not-ok goes into a human review queue rather than into the database.

NLP part:

CAN SAY THAT farmers LOG A TICKET OF THE ISSUE AND WE GIVE IT A CATEGORY/score of severity.

We calculate the severity because of limited engineers/technicians and every farmer says their issue is severe to get it fixed first, or they are so casual/unknowing that their issue is actually serious, or not serious but they are unaware of either way. So we score it.

TF-IDF + logistic regression

TF-IDF (Term Frequency-Inverse Document Frequency) evaluates how important a word is to a document within a corpus.

Measures how often a word appears, TF is normalised by the total words in that document.

The IDF part measures the rarity of the word across the entire corpus, eg words like 'is' or 'the' are everywhere and hence rank low. $\log(N / \text{number of notes containing the term})$. This is large for rare words, near zero for ubiquitous ones.

TF x IDF to filter out generic noise.

Went through all historic notes, collect distinct words and word pairs.

We then count how often keywords pop-up, and they are weighed by how often they come up in the entire corpus of previous tickets that we have. Basically a measure of surprise TF-IDF.

Train 4 logistic regressors, whether the score is ≥ 2 ? ≥ 3 ? ≥ 4 ? ≥ 5 ?

Score = 1 + sum of the four probabilities.

That is the score between 1-5 on how severe.

The OCR is for handwritten notes, not necessarily joined with the NLP.

NLP is for tickets, already text, ranking how severe the ticket might be based on the language.

Cauchy

Changepoint analysis

You have a time series and suspect that the process generating it changed at some point, not that a value was unusual but that the rules changed.

Offline (retrospective). You have the whole series and you're asking "where did the breaks occur?" This is the research setting — labelling historical regimes so you can condition a backtest.

Online (sequential). Data arrives one point at a time and you must decide now whether a change has occurred, as quickly as possible, without too many false alarms.

CUSUM is cumulative sum, accumulating evidence and detecting for an upward shift:

$$S_t^+ = \max(0, S_{t-1}^+ + (x_t - \mu_0 - k))$$

where μ_0 is the in-control mean and k is a slack parameter (typically half the shift size you care about). Fire an alarm when S_t^+ exceeds a threshold h . Run a mirrored S^- for downward shifts.

h controls the delay/false-alarm trade-off directly — raise it and you get fewer false alarms but slower detection.

EWMA control charts are the close cousin: exponentially weight recent observations, alarm when the smoothed value leaves control limits. Similar behaviour, one fewer parameter.

Bayesian Online Changepoint Detection (BOCPD)

You maintain a distribution over the run length — how long since the last changepoint. At each step you update: either the run continues (run length +1) or a change occurs (run length resets to 0), with a hazard rate governing the prior probability of a change. Bayes gives you $P(\text{run length} = r \mid \text{data so far})$.

Slightly expanded:

Asking how long has the current regime lasted, run length.

If no change occurs at time t , then $r_t = r_{t-1} + 1$ (the run grows).

If a change occurs, then $r_t = 0$ (the clock resets).

You don't observe r_t , so you maintain a probability distribution around it. Eg 70% chance the current regime is 45 steps old.

The hazard function $H(r)$. Your prior on how likely a change is, given the regime has already lasted r steps. The simplest choice is a constant: $H(r) = 1/\lambda$, meaning changepoints arrive as a memoryless (geometric) process with expected regime length λ . Set $\lambda = 250$ and you're saying "I expect a regime change roughly once a year." You can make it non-constant so that long-lived regimes are more likely to break.

The predictive model. Given the data in the current run, what do you expect the next observation to look like? For returns, the natural choice is Gaussian with unknown mean and variance, with a Normal-Inverse-Gamma prior — chosen because it's conjugate, which means the posterior update is a closed-form arithmetic operation rather than an integral. That's what makes the whole thing run in real time.

At each new observation x_t , every hypothesis about the run length faces a fork:

Growth. The regime continues. Mass at run length r moves to $r+1$, weighted by:

the probability the regime didn't break: $(1 - H(r))$

how well x_t fits the parameters estimated from that run's data: the predictive likelihood $P(x_t \mid \text{run of length } r)$

Changepoint. The regime breaks. Mass from every run length collapses into $r = 0$, weighted by:
the hazard $H(r)$
the predictive likelihood under a fresh run (i.e. the prior, having seen no data)

"I think the regime started r steps ago, so let me estimate the mean and variance from those r observations, and predict the next point from that."

Now suppose oil has been quiet for 200 days and suddenly moves 6 sigma. The hypothesis " $r = 200$ " assigns that observation a terrible likelihood — under the parameters fitted to 200 calm days, a 6-sigma move is nearly impossible. Meanwhile the hypothesis " $r = 0$ " (fresh regime, wide prior, no committed view on the parameters) assigns it a much higher likelihood, because it hasn't ruled anything out yet.

So Bayes automatically shifts mass to $r = 0$. The changepoint is detected not because you set a threshold, but because the old model became a bad explanation of the data and a fresh model became a better one.

And the reverse: if that 6-sigma move is followed by calm days again, the " $r = 0$ " hypothesis, now fitting one wild point plus calm ones, predicts poorly, while the long-run hypothesis (which never actually died, just lost mass) recovers.

The price you pay: you must specify the hazard rate λ and the prior. λ is the important one, and it's a genuine modelling choice about regime persistence.

Fat tails will hurt you. A Gaussian predictive model will flag every fat-tailed day as a potential regime change. Use a Student-t predictive distribution instead — it's still tractable and vastly more robust.

Vol clustering. GARCH effects look like regime changes to a model that assumes constant variance within a run. Either run BOCPD on GARCH-standardised residuals.

And the unavoidable one: it is still lagging. You need evidence to accumulate before the posterior moves. λ sets how much evidence, and thus the delay/false-alarm trade-off.

--

For retrospective work. You define a cost function (e.g. negative log-likelihood of a Gaussian with a given mean and variance) and search for the set of breakpoints minimising total cost plus a penalty per breakpoint (or you'd just put a breakpoint everywhere).

Binary segmentation: greedily find the single best split, recurse into each half. Fast, approximate.

PELT (Pruned Exact Linear Time): finds the exact optimum in roughly linear time via dynamic programming with pruning.

Markov regime-switching (Hamilton)

Rather than detecting a break, you posit that the series switches between K latent states (e.g. low-vol/high-vol, or trending/mean-reverting), each with its own parameters, governed by a transition matrix. Fit by EM/maximum likelihood; the output is a smoothed probability of being in each state at each time.

Run them on log prices, realised vol, rolling corr or beta and the residual from a fundamental fair-value model. Vol changes are far easier to predict than mean changes.

===

For sizing, just talk about the current model with CTA LOL, risk budgeting.